第5章 基于产生式规则的机器推理

5.1 产生式规则

*5.2 产生式系统

**5.2.1 产生式系统的组成结构

5.2.2 产生式系统的运行

5.2.3 产生式表示的特点及举例

**5.2.4 产生式系统的推理

5.2.5 产生式系统的分类

5.3 产生式系统与图搜索

5.4 基于规则的专家系统

5.1 产生式规则

5.1.1 产生式规则

- 产生式(Production)也称为产生式规则，或简称规则。1943年美国数学家波斯特（E.Post）首先在一种计算形式体系中提出。
- 60年代开始，成为专家系统的最基本的结构(DENTRAL, MYCIN, PROSPECTOR)

- 产生式规则是表示知识的一种方法，适合表示知识单元中的因果关系。产生式的一般形式为：P→Q　或　If P then Q
- 产生式规则的语义:

　　如果前提P被满足，则可推出结论Q或执行Q所规定的动作。即后件由前件来触发。所以，前件是规则的执行条件，后件是规则体。

例如，下面就是几个产生式规则：

(1) 如果银行存款利率下调，那么股票价格上涨。

(2) 如果炉温超过上限，则立即关闭风门。

(3) 如果键盘突然失灵，且屏幕上出现怪字符，则是病毒发作。

(4) 如果胶卷感光度为200，光线条件为晴天，目标距离不超过5米，则快门速度取250，光圈大小取f16。

(5) 如果病人有红色斑点，且病人发烧，且病人是学龄儿童，则病人患的是水痘。

（6） 如果病人出现如打喷嚏、咳嗽等症状，则该病人可能感冒了。

为了严格地描述产生式，下面用BNF范式 (Backus Normal Form) 给出它的形式描述及语义:

〈产生式〉 ::= 〈前提〉→〈结论〉

〈前　提〉 ::= 〈简单条件〉 | 〈复合条件〉

〈结　论〉 ::= 〈事实〉 | 〈操作〉

〈操　作〉 ::= 〈操作名〉[(〈变元〉, …)]

〈复合条件〉::= 〈简单条件〉*AND*〈简单条件〉[(*AND*〈简单条件〉) …] | 〈简单条件〉*OR*〈简单条件〉[(*OR*〈简单条件〉) …]

如MYCIN系统中典型规则定义：

- <rule>=(IF <antecedent> THEN <action> (ELSE <action>))
- (antecedent>=(AND {<condition>})
- <condition>=(OR {<condition>}| (<predicate> <associative_triple>)
- <associative-triple>=(<attribute><object><value>)
- <action>={<consequent>}|{<procedure>}
- <consequent>=(<associative-triple><certainty-factor>)

例：（细菌，染色斑，革兰氏阳性）∧（细菌，形态，球状）∧（细菌，生长结构，链形）→ （细菌，类别，链球菌）

产生式规则与逻辑蕴含式的区别与联系

- 蕴涵式只能表示精确知识,其真值或为真或为假,而产生式不仅可以表示精确知识,还可以表示不精确知识.
- 用产生式表示知识的系统中,决定一条知识是否可用的方法是检查当前是否有已知事实可与前提中所规定的条件匹配,而且匹配可以是精确的,也可以是不精确的,只要按某种算法求出的相似度落在某个预先指定的范围内就认为是匹配的,但对逻辑蕴涵式来说,其匹配总要求是精确的。

5.1.2 基于产生式的推理模式

- 利用产生式规则可以实现有前提条件的指令性操作，也可以实现逻辑推理。
- 实现操作的方法是当测试到一条规则的前提条件满足时，就执行其后部的动作。这称为规则被触发或点燃。
- 利用产生式规则实现逻辑推理的方法是当有事实能与某规则的前提匹配（即规则的前提成立）时，就得到该规则后部的结论（即结论也成立）。

实际上，这种基于产生式规则的逻辑推理模式，就是逻辑上所说的假言推理（对常量规则而言）和三段论推理（对变量规则而言）。

产生式规则形式上很简单，但在一定意义上模仿了人类思考的过程。产生式容易描述事实、规则以及他们的不确定性。

例如：如果下雨，就要打伞

天气冷，就要加衣服

善有善报，恶有恶报。

5.2 产生式系统

什么是产生式系统

把一组产生式放在一起，让它们互相配合协同作用，一个产生式生成的结论可以作为另一个产生式的前提，以这种方式求得问题的解决，这就叫**产生式系统**。

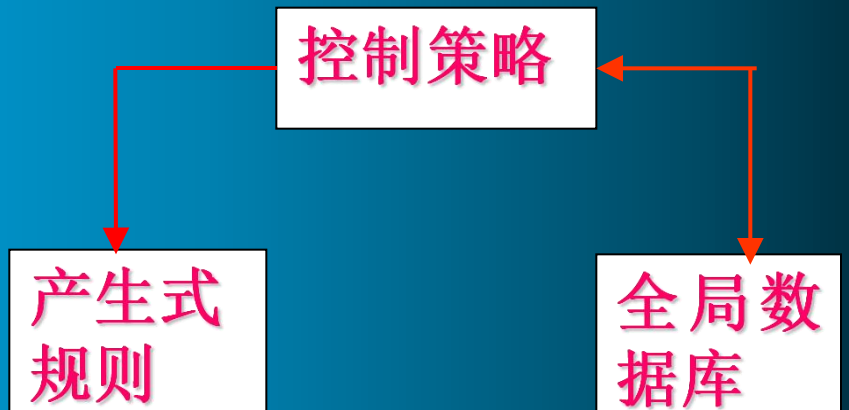
例如：小明很聪明∧小明很努力→小明学习成绩好；

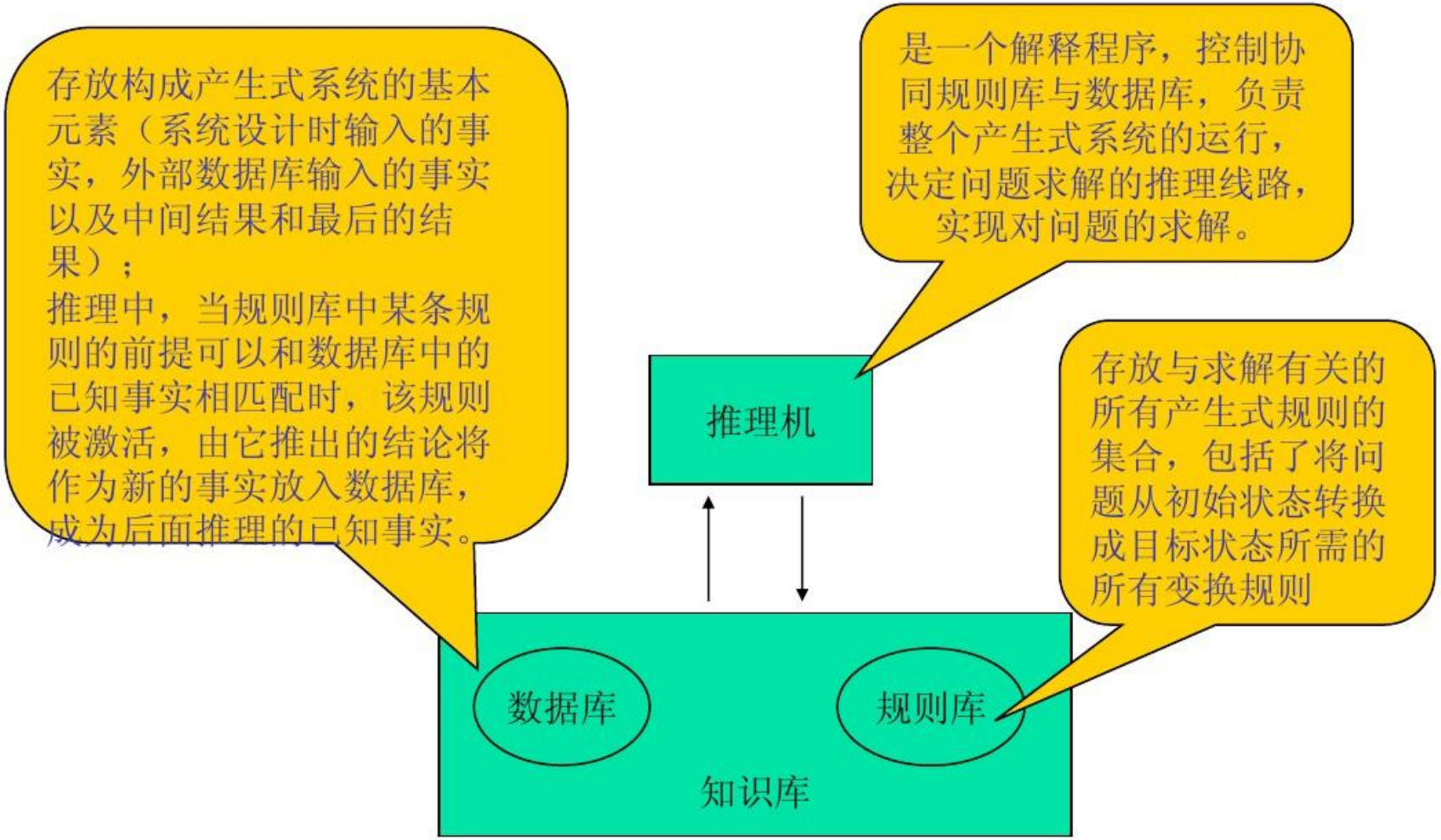
小明学习成绩好→妈妈表扬小明

由于产生式系统的知识库主要用于存储规则，因此它又叫**基于规则的系统(Rule-based System)**。

5.2.1 产生式系统的组成结构

- 组成三要素：
  - 综合数据库——又称事实库、工作区、黑板等。它是一个用于存放问题求解过程中各种当前信息的数据结构，例如问题的初始状态、原始证据、推理中的中间结论及最终结果。全局数据基的内容是不断变化的，是动态的。
  - 产生式规则集——领域知识
  - 控制策略（又称推理机）——规则的解释或执行程序。是一个解释程序，它负责整个产生式系统的运行，用来控制和协调规则集和全局数据基的运行，包含了推理方式和控制策略。它的作用是说明下一步应该选用什么规则，也就是如何应用规则。

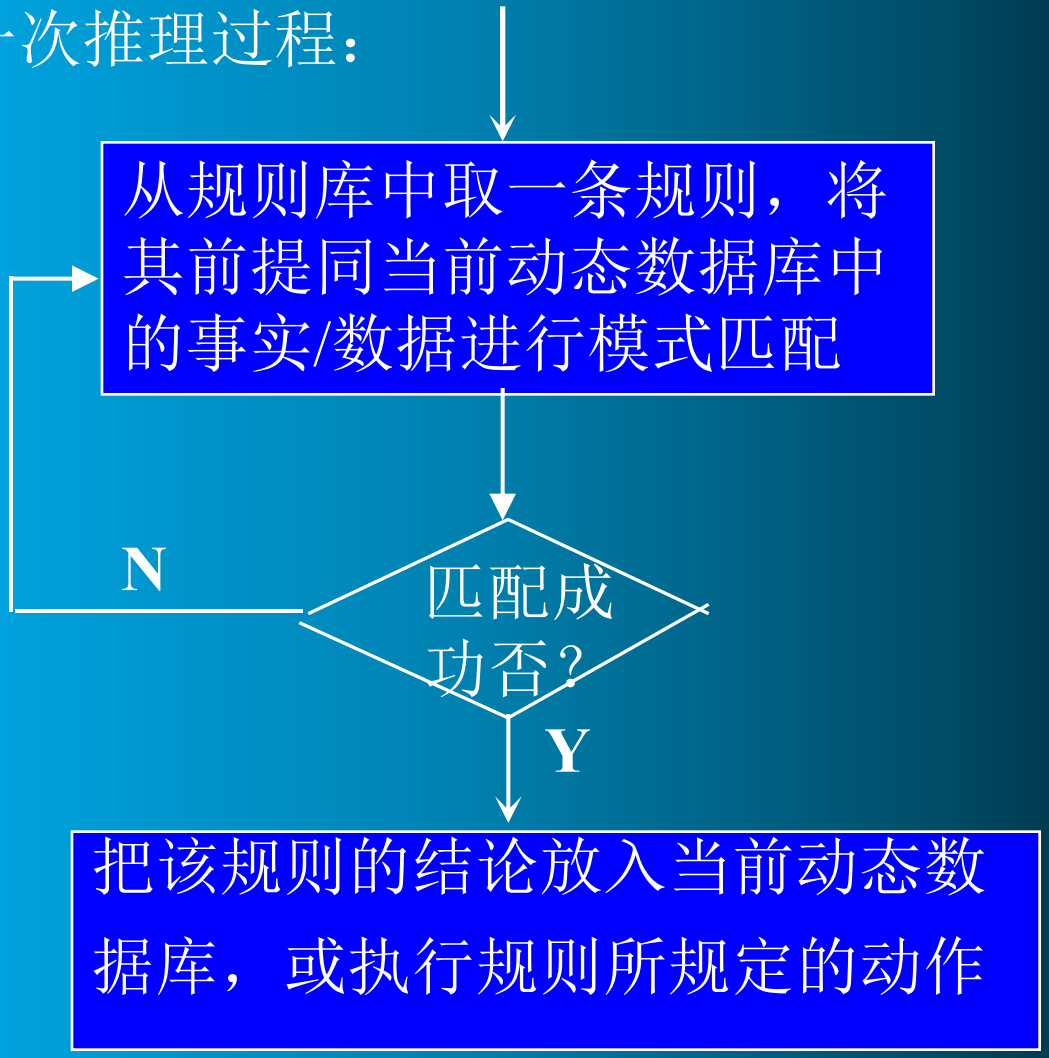



存放构成产生式系统的基本元素（系统设计时输入的事实，外部数据库输入的事实以及中间结果和最后的结果）；
推理中，当规则库中某条规则的前提可以和数据库中的已知事实相匹配时，该规则被激活，由它推出的结论将作为新的事实放入数据库，成为后面推理的已知事实。
是一个解释程序，控制协同规则库与数据库，负责整个产生式系统的运行，决定问题求解的推理线路，实现对问题的求解。
存放与求解有关的所有产生式规则的集合，包括了将问题从初始状态转换成目标状态所需的所有变换规则
推理机
数据库
规则库
知识库

产生式系统的结构图

5.2.2. 产生式系统的运行过程

推理机的一次推理过程：



1. 匹配

按一定的策略从规则集选择规则与全局数据基中的已知事实进行匹配。所谓匹配是指把规则的前提条件与数据基中的已知事实进行比较，如果两者一致或近似一致且满足预先规定的条件，则称匹配成功，相应的规则可被使用。

常用下述方法提高匹配的效率：

（1）索引匹配。为状态(或目标)建立可用产生式索引表，减少可用产生式的搜索范围；

（2）分层匹配。将产生式分成若干层或组，按一定特征分层进行搜索；

（3）过滤匹配。边匹配边按某些附加特征或参数对可用产生式进行精选。

2. 冲突解决

匹配成功的规则可能不止一条，这称发生了冲突。此时推理机必须调用相应的策略进行消解，以便从中选出一条执行。冲突解决策略有很多，常用以下方法进行选择：

- 按规则编排的顺序：把所有产生式排序，选最早匹配成功的一个，不管其余的产生式；
- 规模排序：在所有匹配成功的产生式中取最强者，即前提条件最多或情况元素最多者；
- 就近排序：最近用过的产生式优先(或反之)；
- 数据排序（按优先级）：给情况以不同的优先权，与优先权较高的情况相匹配的产生式优先使用；
- 使用估计函数f(x)排序；
- 上下文排序：利用上下文限制。
- 专一性排序（按针对性）

3. 操作

操作就是执行规则的右部，经过操作以后，当前数据基将被修改，然后其它的规则有可能被使用。

对于不确定性知识，在执行每一条规则时还要按一定算法计算结论的不确定性。

4. 随时掌握结束产生式系统运行的时机，以便在适当的时候停止系统的运行。

- 以上各点中的每一项都有许多工作要做，在此不做详细讨论。

产生式系统求解问题的一般步骤

1，初始化全局数据基，把问题的初始已知事实送入数据基中

2，若规则库中存在尚未使用过的规则，而且它的前提可与数据基中的已知事实匹配，则转3步，若不存在这样的事实，则转5步。

3，执行当前选中的规则，并对该规则做上标记，把该规则执行后的结论送入数据基中。如果该规则的结论是某些操作，则执行之。

4，检查数据基中是否已包含了问题的解，若已包含，则终止问题的求解过程，否则转2步；

5，要求用户提供进一步的关于问题的已知事实，若能提供，则转2步，否则终止问题的求解过程。

6，若规则库中不再有未使用过的规则，则终止求解过程。

产生式系统求解问题的一般步骤（补充）

一般的产生式系统可用下述基本过程来描述：

步1 DATA←初始数据库；

步2 until DATA满足终止条件，do；

步3 begin

步4 在规则集中选择能作用到DATA上的规则R；

步5 DATA←R作用到DATA上的结果；

步6 end。

这是一个非确定性的程序。其中在步4中没有具体说明如何选择一个适当的规则。

规则的模式匹配

规则引擎的核心是Pattern Matcher(模式匹配器)。不管是正向推理还是反向推理，首先要解决一个模式匹配的问题。规则的模式匹配可以定义为： 一个规则是一组模式的集合。如果事实/假设的状态符合该规则的所有模式，则称为该规则是可满足的。 模式匹配的任务就是将事实/假设的状态与规则库中的规则一一匹配，找到所有可满足的规则。

假设系统中有N条规则，平均每个规则的条件部分(LHS)有P个模式，在某个时点有M个事实需要处理。则规则匹配要做的事情就是： 对每一个规则r，判断当前的事实o是否满足LHS(r)=True，如果满足，则将规则r的实例r(o)，即规则+满足该规则的事实，加到冲突集中等待处理。 通常采取如下过程：

- 从N条规则中取出一条r；
- 从M个事实中取出P个事实的一个组合c；
- 用c测试LHS(r)，如果LHS(r（c）)=True，将事实部分RHS(r（c）)加入队列中；
- 如果M个事实还存在其他的组合c，goto 3；
- 取出下一条规则r，goto 2；

◆关于如何选择可用规则的算法有（补充）

上面的处理由于涉及到组合，过程很复杂。有必要通过特定的算法优化匹配的效率。目前常见的模式匹配算法包括Markov、Rete、Treat、Leaps，HAL，Matchbox等。

- **Markov算法**

  按优先级对规则集合排序，每次选择优先级最高的规则应用。但是当规则集合比较大时，该算法比较低效，高效的算法应当避免每次都检查所有的规则。

- **Rete算法**

Rete模式匹配算法

Rete算法是目前使用最广泛的规则匹配算法，由Charles L. Forgy博士在1979年发明。Rete算法是一种快速的前向链推理算法，其匹配速度与规则的数量无关。

Rete的高效率主要来自两个重要的假设：

- **时间冗余性**。facts在推理过程中的变化是缓慢的, 即在每个执行周期中,只有少数的facts发生变化，因此影响到的规则也只占很小的比例。所以可以只考虑每个执行周期中已经匹配的facts.
- **结构相似性**。许多规则常常包含类似的模式和模式组。

该算法是一个快速的模式匹配算法，它通过存储关于规则的信息而获得速度。在每个“识别—动作”循环中，Rete算法不是检查所有的规则，**而是只检查匹配情况发生变化的那些规则，由于在每个周期一般只有少量数据发生变化，因此忽略那些没有变化的数据将大大加快规则匹配的速度。**

- Rete算法的基本思想是保存过去匹配过程中留下的全部信息,以空间代价来换取执行效率 。对每一个模式 ,附加一个匹配元素表来记录WorkingMemory中所有能与之匹配的元素。当一个新元素加入到WorkingMemory时, 找出所有能与之匹配的模式, 并将该元素加入到匹配元素表中; 当一个元素从WorkingMemory中删除时,同样找出所有与该元素匹配的模式, 并将元素从匹配元素表中删除。 Rete算法接受对工作存储器的修改操作描述 ,产生一个修改冲突集的动作 。

Rete算法的步骤如下：

- *将初始数据（fact）输入Working Memory。*
- *使用Pattern Matcher比较规则（rule）和数据（fact）。*
- *如果执行规则存在冲突（conflict），即同时激活了多个规则，将冲突的规则放入冲突集合。*
- *解决冲突，将激活的规则按顺序放入Agenda。*
- *使用规则引擎执行Agenda中的规则。重复步骤2至5，直到执行完毕所有Agenda中的规则。*

产生式系统的特点

- 自然、直观，是最常用的知识表示形式
- 数据驱动，知识的无序性
- 控制系统与问题无关
- 数据、知识和控制相对独立（模块化）

缺点是

- 效率不高

  产生式系统求解问题的过程是一个反复进行“匹配--冲突消解—执行”的过程。鉴于规则库一般都比较庞大，而匹配又是一件十分费时的工作，因此其工作效率不高。

- 不能表示具有结构性的知识

5.2.3 问题表示举例（补充）

例1：设字符转换规则

A∧B→C

A∧C→D

B∧C→G

B∧E→F

D→E

已知：A，B

求：F

一、综合数据库

{x}，其中x为字符

二、规则集

1，IF A∧B THEN C

2，IF A∧C THEN D

3，IF B∧C THEN G

4，IF B∧E THEN F

5，IF D THEN E

三、控制策略

顺序排队

四、初始条件 {A，B}

五、结束条件 F∈{x}

求解过程

数据库	可触发规则	被触发规则
A，B	（1）	（1）
A，B，C	（2） （3）	（2）
A，B，C，D	（3） （5）	（3）
A，B，C，D，G	（5）	（5）
A，B，C，D，G，E	（4）	（4）
A，B，C，D，G，E，F		

1，IF A∧B THEN C
2，IF A∧C THEN D
3，IF B∧C THEN G
4，IF B∧E THEN F
5，IF D THEN E

例2：传教士与野人问题（M-C问题）

问题：N个传教士，N个野人，一条船，可同时乘坐k个人，要求在任何时刻，在河的两岸，传教士人数不能少于野人的人数。

问：如何过河。以N=3，k=2为例求解。

左岸

	L	R
m	3	0
C	3	0
B	1	0



右岸

	L	R
m	0	3
C	0	3
B	0	1

1，综合数据库

（m, c, b)，其中：0≤m, c≤3, b ∈{0, 1}

2，初始状态 （3，3，1）

3，目标状态（结束状态） （0，0，0）

M-C问题（续）

4，规则集

IF (m, c, 1) THEN (m-1, c, 0)
IF (m, c, 1) THEN (m, c-1, 0)
IF (m, c, 1) THEN (m-1, c-1, 0)
IF (m, c, 1) THEN (m-2, c, 0)
IF (m, c, 1) THEN (m, c-2, 0)

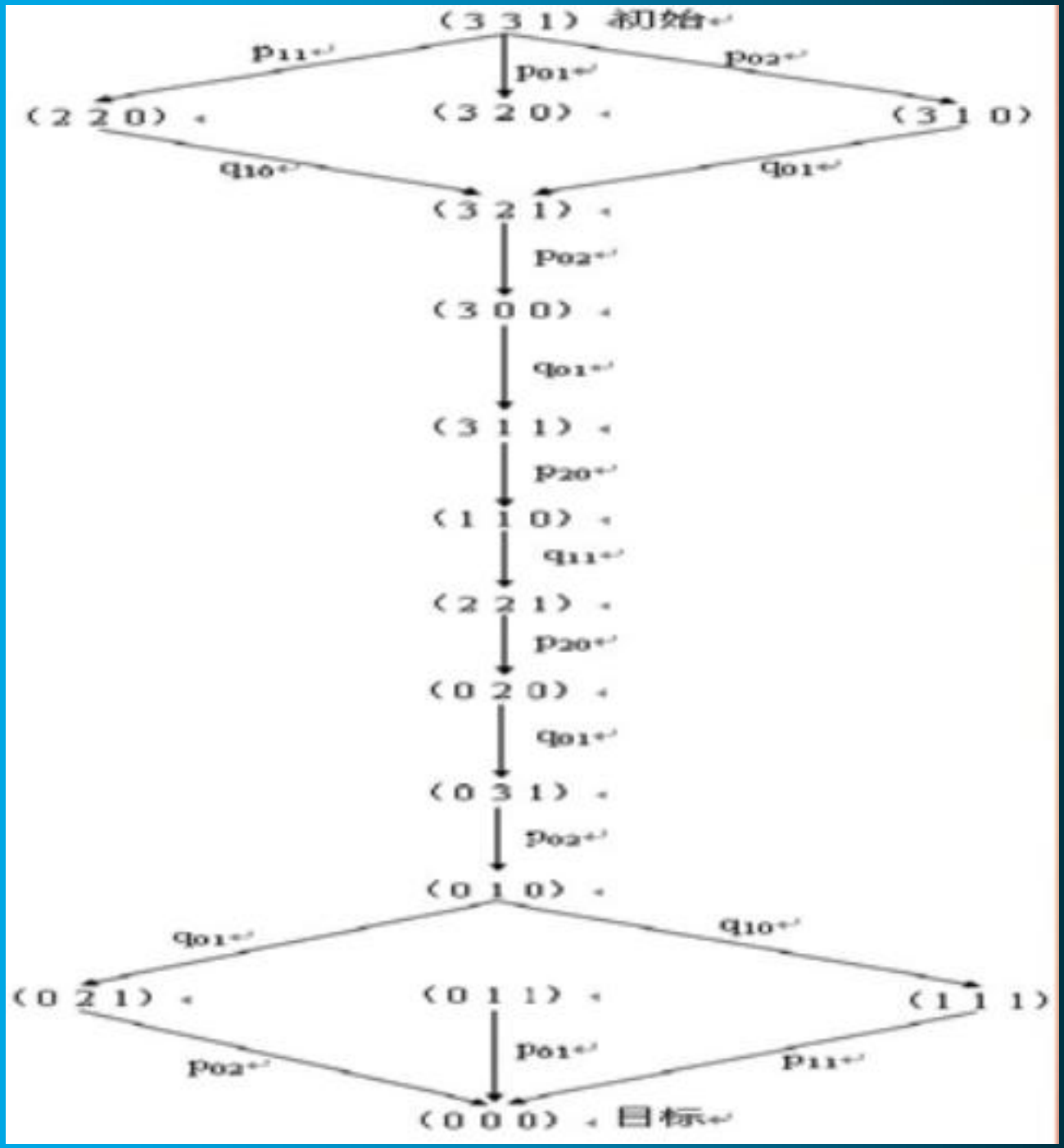
IF (m, c, 0) THEN (m+1, c, 1)
IF (m, c, 0) THEN (m, c+1, 1)
IF (m, c, 0) THEN (m+1, c+1, 1)
IF (m, c, 0) THEN (m+2, c, 1)
IF (m, c, 0) THEN (m, c+2, 1)

5，控制策略：（略）

M-C问题（另一种表示）

4，规则集：

IF (m, c, 1) AND 1 ≤i+j≤2 THEN (m-i, c-j, 0)
IF (m, c, 0) AND 1 ≤i+j≤2 THEN (m+i, c+j, 1)


(3 3 1) 初始
p11
p01
p02
(2 2 0)
(3 2 0)
(3 1 0)
q10
q01
(3 2 1)
p02
(3 0 0)
q01
(3 1 1)
p20
(1 1 0)
q11
(2 2 1)
p20
(0 2 0)
q01
(0 3 1)
p02
(0 1 0)
q01
q10
(0 2 1)
(0 1 1)
(1 1 1)
p02
p01
p11
(0 0 0) 目标

例3. 猴子摘香蕉问题

1，综合数据库

（M, B, Box, On, H）

M：猴子的位置

B：香蕉的位置

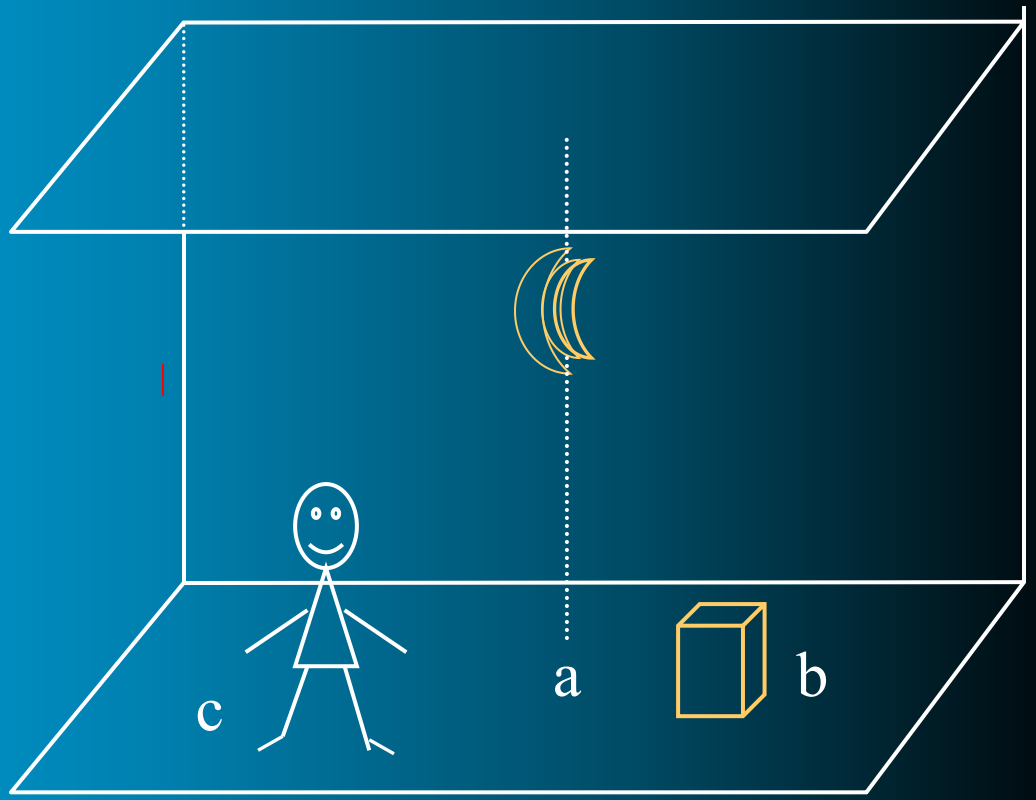
Box：箱子的位置

On=0：猴子在地板上,

On=1：猴子在箱子上

H=0：猴子没有抓到香蕉,

H=1：猴子抓到了香蕉



2，初始状态 （c, a, b, 0, 0）

3，结束状态 （x1, x2, x3, x4, 1）其中x1～x4为变量。

猴子摘香蕉问题（续2）

4，规则集

r1: IF (x, y, z, 0, 0) THEN (w, y, z, 0, 0)

r2: IF (x, y, x, 0, 0) THEN (z, y, z, 0, 0)

r3: IF (x, y, x, 0, 0) THEN (x, y, x, 1, 0)

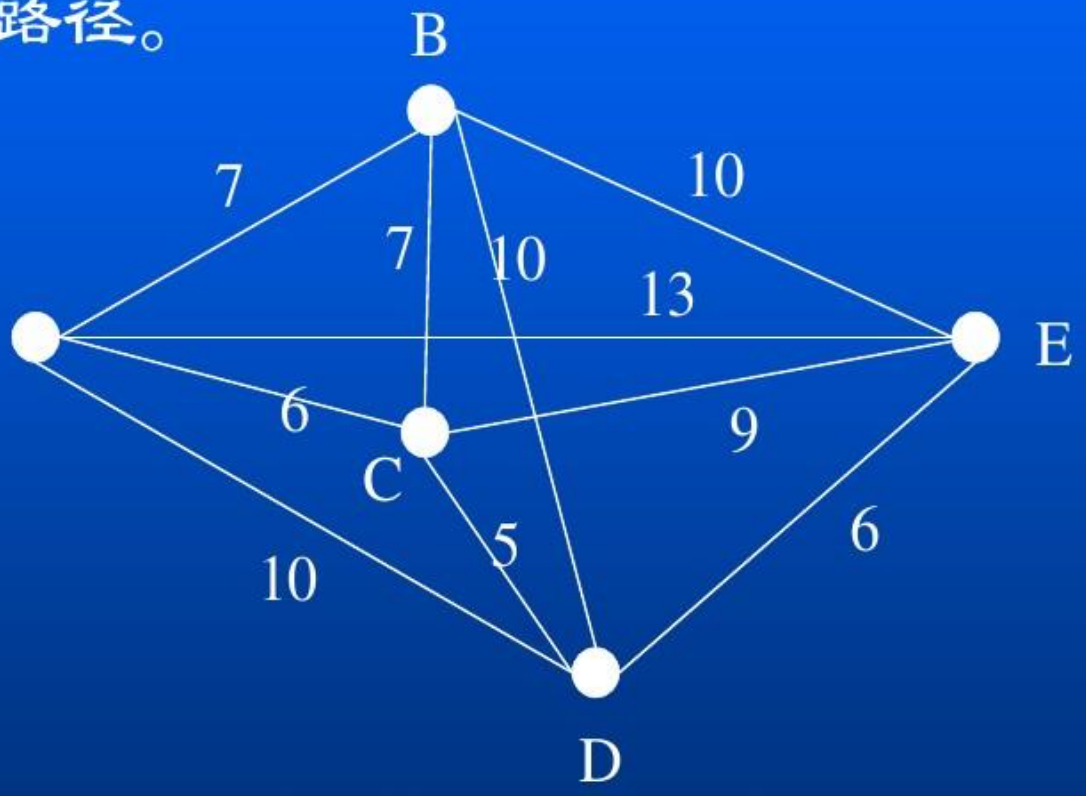
r4: IF (x, y, x, 1, 0) THEN (x, y, x, 0, 0)

r5: IF (x, x, x, 1, 0) THEN (x, x, x, 1, 1)

其中x, y, z, w为变量

旅行商问题：一个推销员要到几个城市去办理业务，城市间里程数已知，问题的提法是：从某个城市出发，每个城市只允许访问一次，最后又回到最初出发的城市，求一条最短距离的路径。

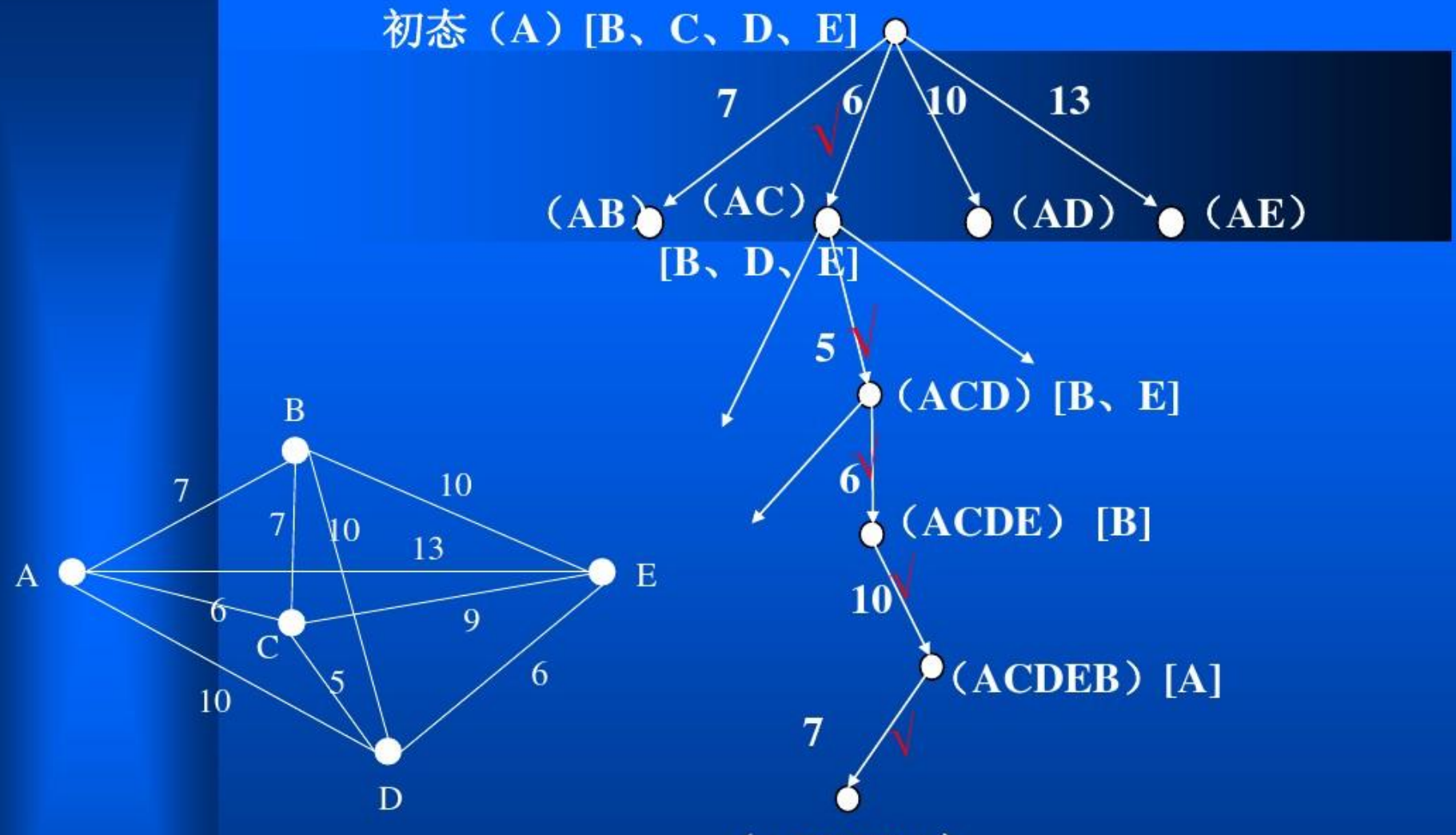
- 5个城市旅行商问题的地图（全连图）如图所示，求从A A出发经B、C、D、E再回到A的最短路径。



旅行商问题的地图

问题的产生式描述：

- 综合数据库：若每个城市用一个字母表示，则问题状态可用一个字母组成的表或字符串来表示，如（A）表示初始状态，（A****A）表示目标状态，（A**）表示访问两个城市后的当前状态。
- 规则集：1）下一步走向城市A；2）下一步走向城市B；…，…5）下一步走向城市E；
- 问题的约束：不重复走到同一城市，即在没有转完所有城市时，不能走向出发城市。
- 控制策略：每次走向离的最近的城市（启发信息）



用图搜索生成的部分搜索树

Artificial Intelligence

【例5】某甲要去参加一项智力竞赛，他风闻另外两名对手都是聪明人，心中十分担忧。幸好，他的一位朋友打听到了竞赛题目，并把应付办法事先告诉了他。

竞赛开始，主持人在每个竞赛者头上戴一顶帽子，帽子的颜色分红白两种，但至少有一顶是白帽。题目是说出自己所戴帽子的颜色。戴毕，主持人连问两次，三人面面相觑，无一人能答。问到第三次时，某甲抢先给出了答案。试问朋友面授某甲的妙计是什么？

它可以表成如下的谓词产生式系统：

- 帽色(聪明人x, 红)∧帽色（聪明人y, 红）∧ x≠y → 帽色（自己，白）
- 帽色(聪明人x, 红）∧帽色（聪明人y, 白）∧答不出（聪明人y）→帽色（自己，白）
- 帽色(聪明人x, 红）∧帽色（聪明人y, 白）∧答出（聪明人y）→帽色（自己，红）
- 帽色(聪明人x, 白）∧帽色（聪明人y, 白）∧答不出（聪明人y）∧又答不出（聪明人y）→帽色（自己，白）
- 帽色(聪明人x, 白）∧帽色（聪明人y, 白）∧答不出（聪明人y）∧第二次答出（聪明人y）→帽色（自己，红）

思考题： 三个聪明人问题。

古代有个国王想知道他的三个大臣中谁最聪明，就在他们每个人的前额上画了一个点，国王说，你们中间至少有一个人的点是白色的。于是重复地问他们：“谁知道自己点的颜色？”三位大臣头两次都回答说不知道，试证明下一次他们都会说“知道”，而且所有的点都是白色的。

5.2.4 产生式系统的推理

可分为正向推理和反向推理两种基本方式。

下面我们给出产生式系统正向推理和反向推理的常用算法：

1.产生式系统的控制策略（正向推理）

正向推理是指：从初始事实/数据出发，正向使用规则进行推理，朝目标方向前进。

正向推理步骤

1、正向推理步骤

① 读入初始数据（事实）集到工作存储器,待测试规则表清空

② 寻找与初始事实相匹配的规则。取出规则，将规则的全部前件与工作器中所有的事实比较。如匹配成功，将规则加入冲突集；如冲突集为空，转向（3）；否则冲突消解；将所选择的规则的结论加入工作存储器；如达到目标结点转向（3）；否则返回（2）。

③ 结束

正向推理特点

- 在上面的例子中，推理机给出的推理结果是“苹果”
- 正向推理的特点：
  - 正向推理是数据驱动，从一组事实出发推导结论。
  - 优点是：算法简单，容易实现，允许用户一开始就把有关的事实存入数据库
  - 缺点是：盲目搜索，可能会求解许多与总目标无关的子目标，推理效率底。

正向推理算法：

步1 将初始事实/数据置入动态数据库；

步2 用动态数据库中的事实/数据，匹配/测试目标条件，若目标条件满足，则推理成功，结束。

步3 用规则库中各规则的前提匹配动态数据库中的事实/数据，将匹配成功的规则组成待用规则集；

步4 若待用规则集为空，则运行失败，退出。

步5 用某种**冲突消解策略，**将待用规则集中各规则的结论加入动态数据库，或者执行其动作，转步2；

可以看出，随着推理的进行，动态数据库的内容或者状态在不断变化。但算法并未记录动态数据库的状态变化历史，而是始终保持当前的一个动态数据库状态，同时也始终基于当前数据库进行推理。

例5.1 动物识别系统IDENTIFIER

设由下列动物识别规则组成一个规则库，推理机采用上述正向推理算法，建立一个产生式系统。该产生式系统就是一个小型动物分类知识库系统。规则：

r_1：若某动物有奶，则它是哺乳动物。

r_2：若某动物有毛发，则它是哺乳动物。

r_3：若某动物有羽毛，则它是鸟。

r_4：若某动物会飞且生蛋，则它是鸟。

r_5：若某动物是哺乳动物且有爪且有犬齿且目盯前方，则它是食肉动物。

r_6：若某动物是哺乳动物且吃肉，则它是食肉动物。

r_7：若某动物是哺乳动物且有蹄，则它是有蹄动物。

r_8：若某动物是有蹄动物且反刍食物，则它是偶蹄动物。

r_9：若某动物是食肉动物且黄褐色且有黑色条纹，则它是老虎。

r_{10}：若某动物是食肉动物且黄褐色且有黑色斑点，则它是金钱豹。

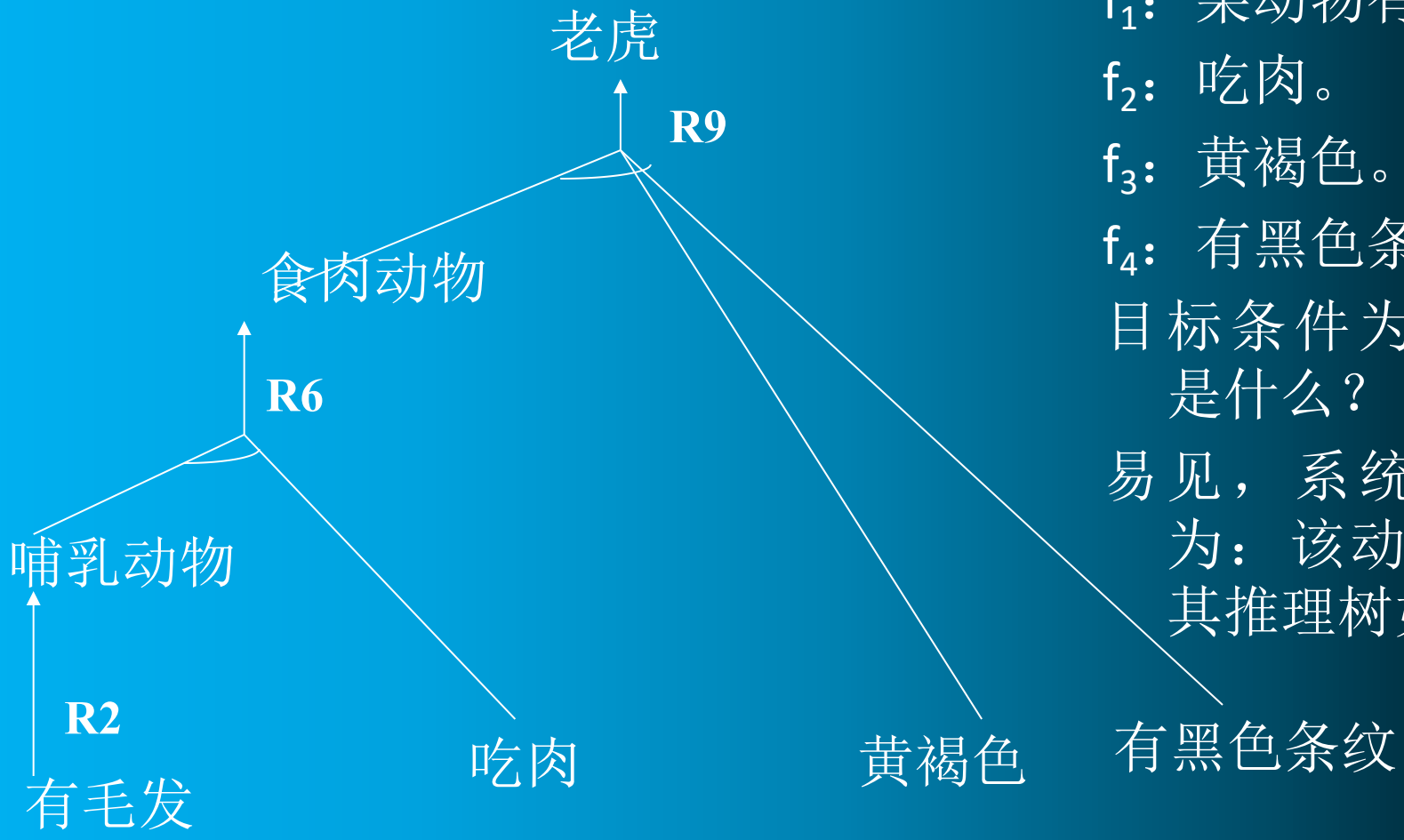
r_{11}：若某动物是有蹄动物且长腿且长脖子且黄褐色且有暗斑点，则它是长颈鹿。

r_{12}：若某动物是有蹄动物且白色且有黑色条纹，则它是斑马。

r_{13}：若某动物是鸟且不会飞且长腿且长脖子且黑白色，则它是驼鸟。

r_{14}：若某动物是鸟且不会飞且会游泳且黑白色，则它是企鹅。

r_{15}：若某动物是鸟且善飞且不怕风浪，则它是海燕。



再给出初始事实：

f_1：某动物有毛发。

f_2：吃肉。

f_3：黄褐色。

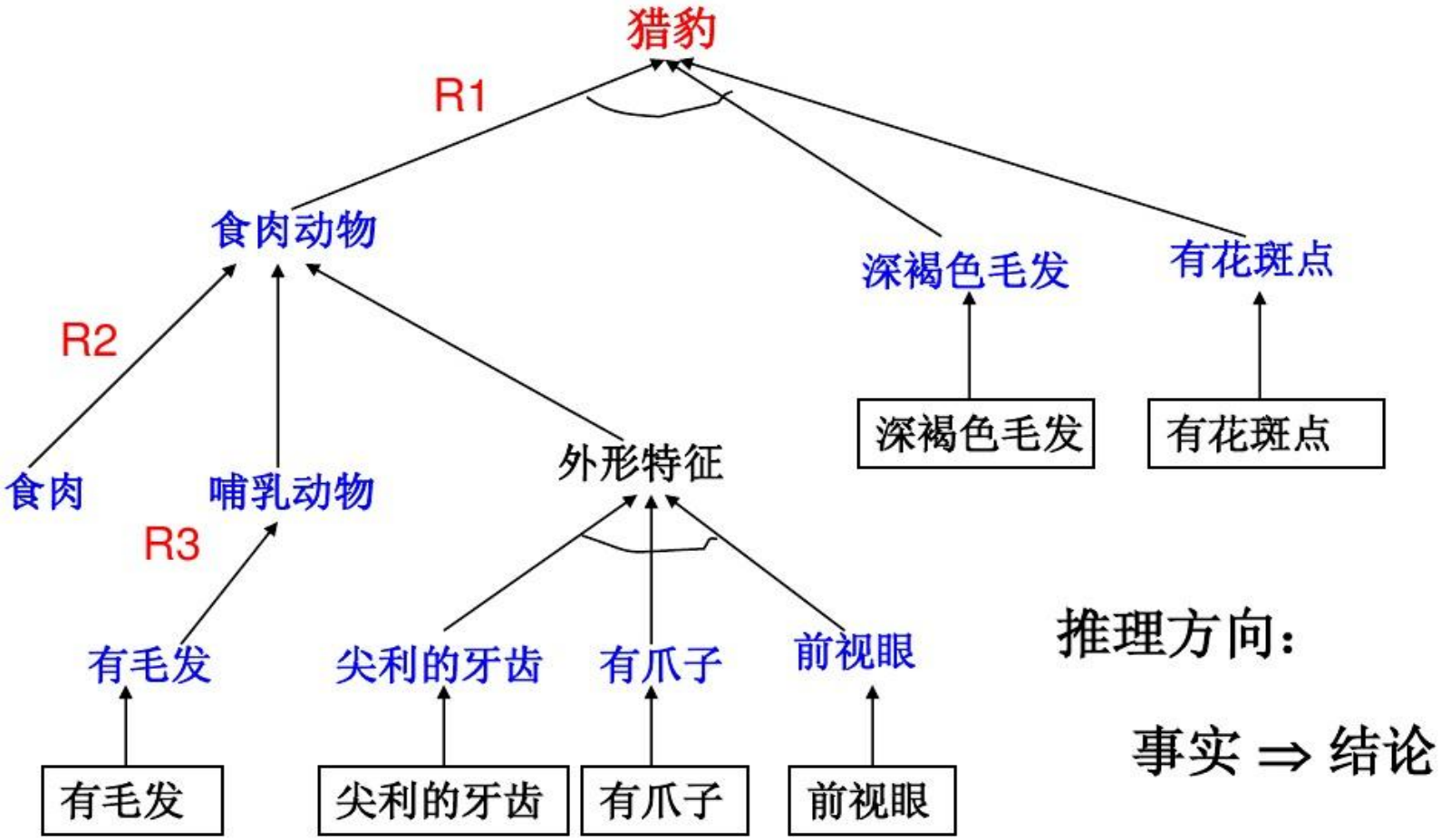
f_4：有黑色条纹。

目标条件为：该动物是什么？

易见，系统运行结果为：该动物是老虎，其推理树如图5—4。

图5-4 动物分类正向推理树

正向推理的产生式系统实例（1）



推理方向：

事实 ⇒ 结论

2. 产生式系统的反向推理

反向推理：从目标出发，反向使用规则进行推理，朝初始事实或数据方向前进。

例2 对于例1中的产生式系统，反向推理树如下图：

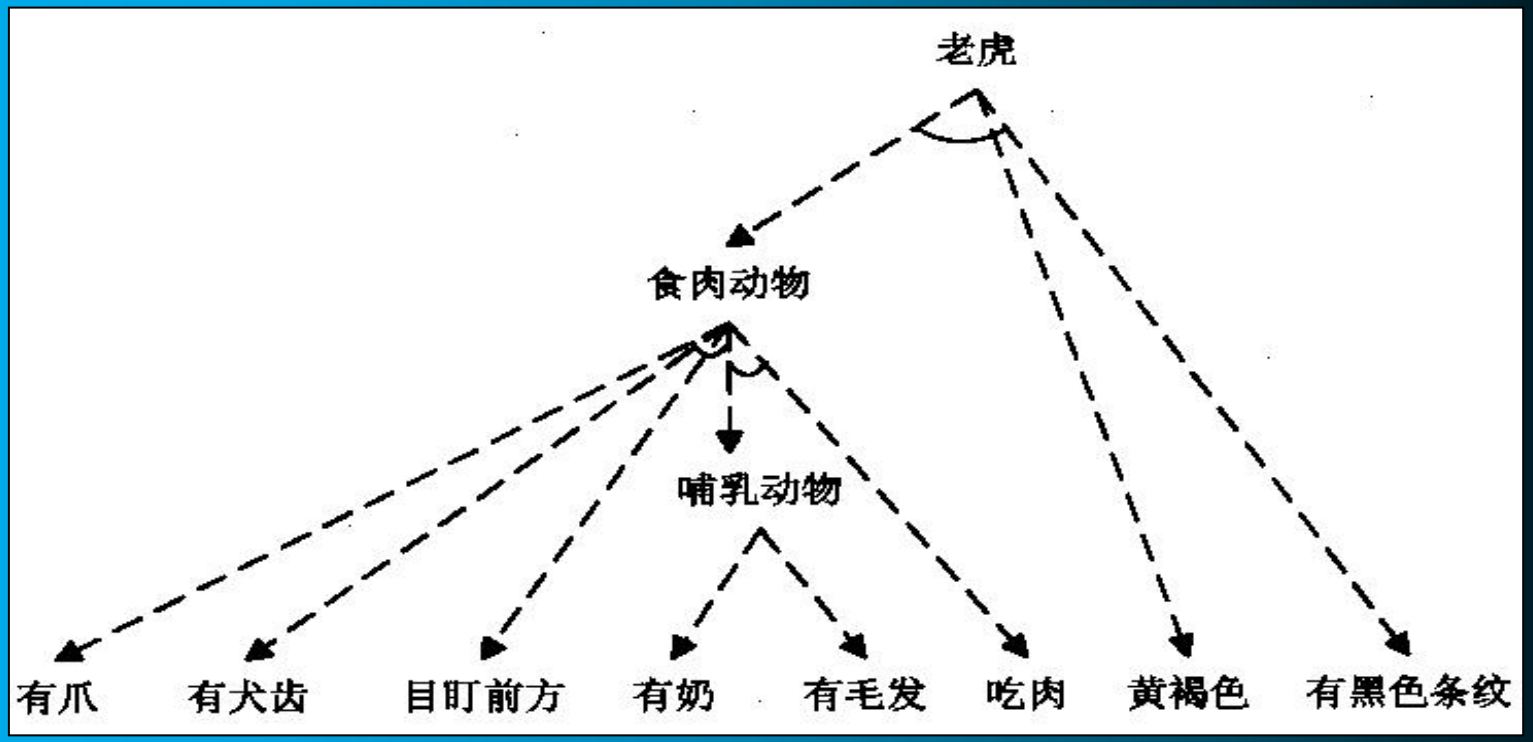
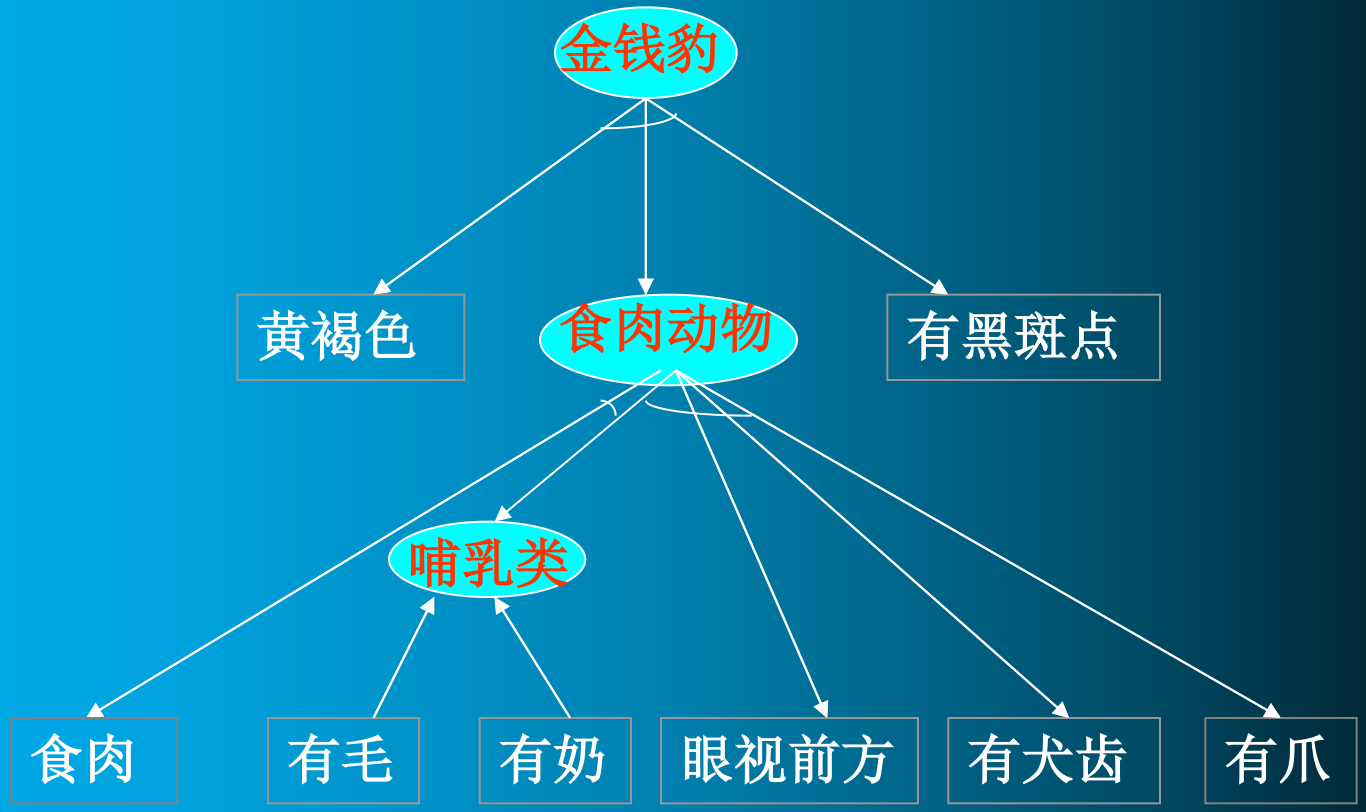


图5—5 动物分类反向推理树


金钱豹
黄褐色
食肉动物
有黑斑点
哺乳类
食肉
有毛
有奶
眼视前方
有犬齿
有爪

- 步1　初始化动态数据库，将初始事实、数据置入动态数据库。将目标条件置入目标链。
- 步2　若目标链为空，则推理成功，结束。
- 步3　取出目标链中第一个目标，用动态数据库中的事实、数据同其匹配，若匹配成功，转步2。
- 步4　用规则库中各规则的结论同该目标匹配，若匹配成功，则将第一个匹配成功且未用过的规则的前提作为新的目标，并取代原来的父目标而加入目标链，转步3。
- 步5　若该目标是初始目标，则推理失败，退出。
- 步6　将该目标的父目标移回目标链，取代该目标及其兄弟目标，转步3。

正（逆）向产生式系统的比较

特点	正向推理	逆向推理
推理驱动方式	数据驱动	目标驱动
优点	算法简单，易于实现	搜索目的性强，推理效率高
缺点	1、搜索目的性弱，可能求解出多个无关的结论； 2、 匹配时，要遍历整个规则库，推理效率低。	1、确定目标的时候，具有盲目性，可能产生假目标 2、当规则的后件是操作而非断言时，即反应型系统，不宜使用此法
用途	主要用于已知初始数据，不知目标的推理；或是**解空间**大的一类推理。	主要用于结论单一或已知目标求证的一类推理。
应用	监控、预测、规划、设计等	选择、分类、故障诊断等

可以看出，与正向推理不同，这次的推理树是从上而下扩展而成的，而且推理过程中还发生过回溯。

反向推理也称为后向推理、反向链、目标驱动的推理等。从上面的两个算法可以看出，正向推理是自底向上的综合过程，而反向推理则是自顶向下的分析过程。

除了正向推理和反向推理外，产生式系统还可进行双向推理。双向推理就是同时从初始数据和目标条件出发进行推理，如果在中间某处相遇，则推理搜索成功。

5.2.5 产生式系统分类

1. 按搜索策略分

① 不可挽回的产生式系统——规则使用后，不允许回过头来重新选用其他规则。

② 试探性的产生式系统——规则使用后，允许返回原出发点重新选用其他规则。试探性的产生式系统又可分为两类：

·回溯式产生式系统——规则使用后，记住原来的结点，如搜索遇到困难时可返回再选其他规则。例如有界深度优先搜索。

·图搜索式产生式系统——它同时掌握若干规则序列的效果，从中寻找问题的答案。为避免循环，通常采用树搜索。例如广度优先搜索。

2. 按搜索方向分

一般是从初始状态向着目标方向进行搜索，如果规则可以逆向运用，也可以从目标状态向着初始状态方向进行搜索，或者双向同时进行搜索。这样就形成了三种不同方向的产生式系统：

向前产生式系统；

向后产生式系统；

双向产生式系统

3. 除了一般类型的产生式系统外，还有两种特殊类型的产生式系统：

① 可交换的产生式系统

各规则的选用次序不重要。

② 可分解的产生式系统

初始数据库可分解为若干独立部分，可分别使用规则，直到目标状态为止。这时终止条件也相应被分解。

1. 回溯式产生式系统

- 对于搜索量小的问题，回溯式策略往往是完备的和有效的，它易于实现，且占用存贮量小。

- 回溯式产生式系统的工作原理可用一个递归过程BACKTRACK来实现。这是一种局部试探性搜索系统。

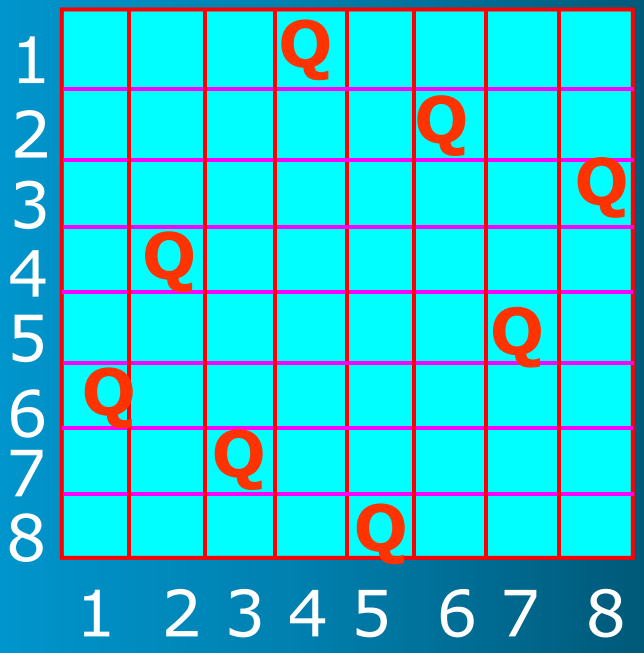
- **Term(DATA)**
  判DATA是否满足终止条件的谓词；
- **DEADEND(DATA)**
  判DATA是否满足失败条件的谓词；
- **APPRULES(DATA)**
  选取可作用在DATA上的所有操作并排序的函数；
- **DATA(A)**
  求操作R作用在DATA上所产生的新数据库的函数；
- **NULL(RULES)**
  判可用规则表空否的谓词
- **FIRST(RULES)**
  取可用规则表第一个元素的函数；
- **TAIL(RULES)**
  在可用规则表中除去第一个元素的函数；
- **CONS(R, PATH)**
  把元素R加入到路径表PATH的最前面的函数

递归过程BACKTRACK(DATA)

步 1　if TERM (DATA), return NIL （按成功返回）；
步 2　if DEADEND (DATA), return FAIL (按失败返回);
步 3　RULES ← APPRULES(DATA);
步 4　LOOP: if NULL(RULES), return FAIL;
步 5　R ← FIRST(RULES);
步 6　RULES ← TAIL(RULES);
步 7　RDATA ← R (DATA);
步 8　PATH ←BACKTRACK(RDATA);
步 9　if PATH = FAIL, go LOOP （重选其他规则);
步10　return CONS (R PATH).

八皇后问题

八皇后问题是一个古老而著名的问题，是回溯算法的典型案例。该问题是国际西洋棋棋手马克斯·贝瑟尔于1848年提出：在8×8格的国际象棋上摆放八个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列或同一斜线上，问有多少种摆法。 高斯认为有76种方案。1854年在柏林的象棋杂志上不同的作者发表了40种不同的解，后来有人用图论的方法解出92种结果。计算机发明后，有多种计算机语言可以解决此问题。



二.问题分析

显然，每一行可以而且必须放一个皇后，所以n皇后问题的解可以用一个n元向量X=（$x_1,x_2,.....x_n$）表示，其中，$1\le i\le n$且$1\le x_i\le n$，即第n个皇后放在第i行第x_i列上。

由于两个皇后不能放在同一列上，所以，解向量X必须满足的约束条件为：$x_i\neq x_j$

若两个皇后的摆放位置分别是（i,x_i）和（j,x_j），在棋盘上斜率为-1的斜线上，满足条件$i-j=x_i-x_j$;在棋盘上斜率为1的斜线上，满足条件$i+j=x_i+x_j$;

综合两种情况，由于两个皇后不能位于同一斜线上，

所以，解向量X必须满足的约束条件为：$|i-x_i|\neq |j-x_j|$

四皇后问题

在n×n格的棋盘上放置彼此不受攻击的n个皇后。按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。n后问题等价于在n×n格的棋盘上放置n个皇后，任何2个皇后不放在同一行或同一列或同一斜线上。

例：4皇后问题

- 全局数据库：

  用A（i，j）（其中1≤i≤4，1≤j≤4）表示出现在位置(i, j)的皇后，用已出现的皇后序列表示全局数据库。

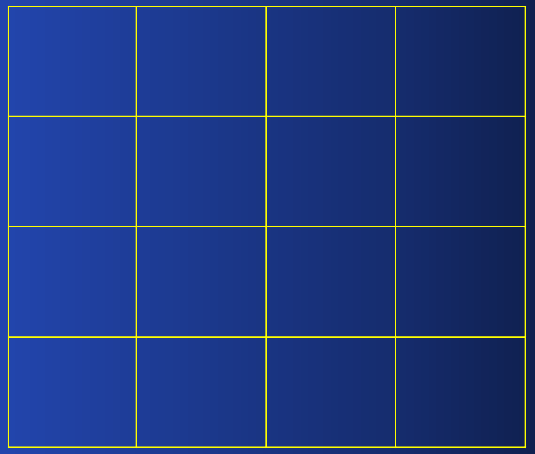
- 规则集：

  $\{R_{ij}\}$（其中1≤i≤4，1≤j≤4）

- 控制系统：

  按自然顺序选择规则。

( )

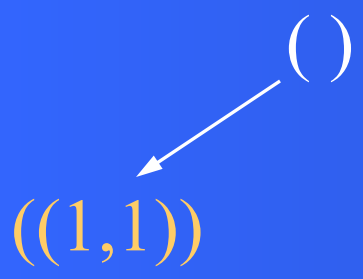


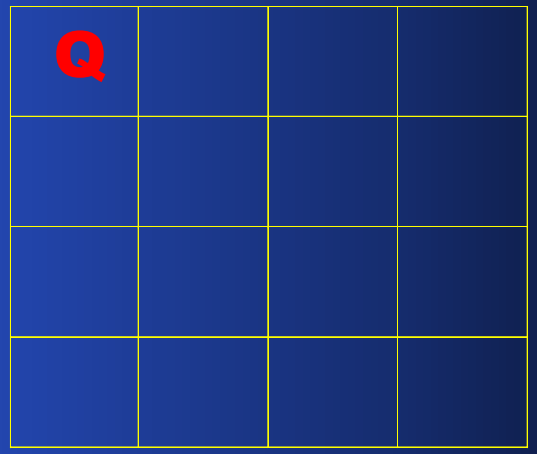
第一层递归:

DATA=NIL, Rules=(R11,R12,R13,R14)

R=R11，Rules’=(R12,R13,R14)

RDATA=(A11)

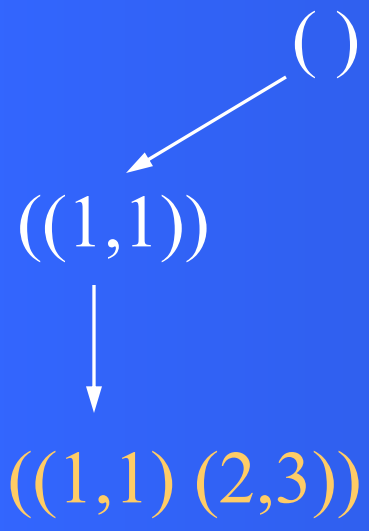




第二层递归:

DATA=(A11), Rules=(R23,R24)

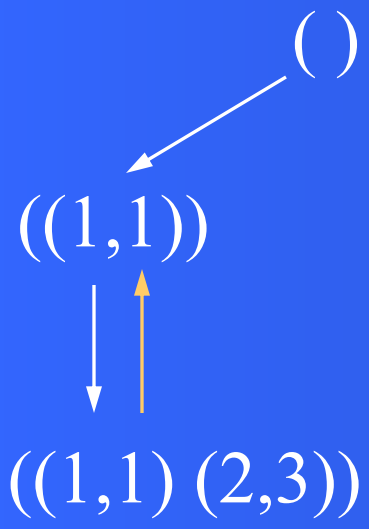
R=R23，Rules’=(R24)　　RDATA=(A11 A23)

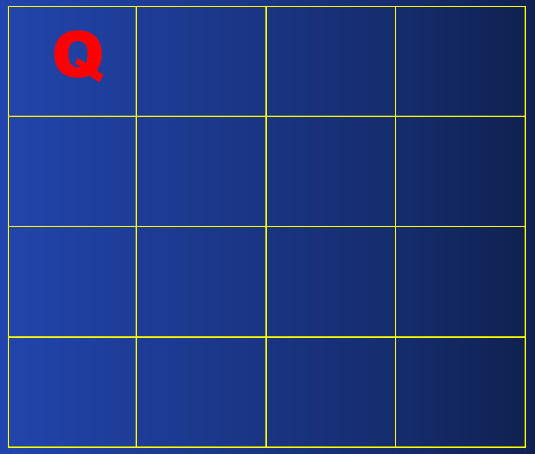


Q			
		Q	

第三层递归:

DATA=(A11 A23), Rules=NIL失败,返回第二层递归



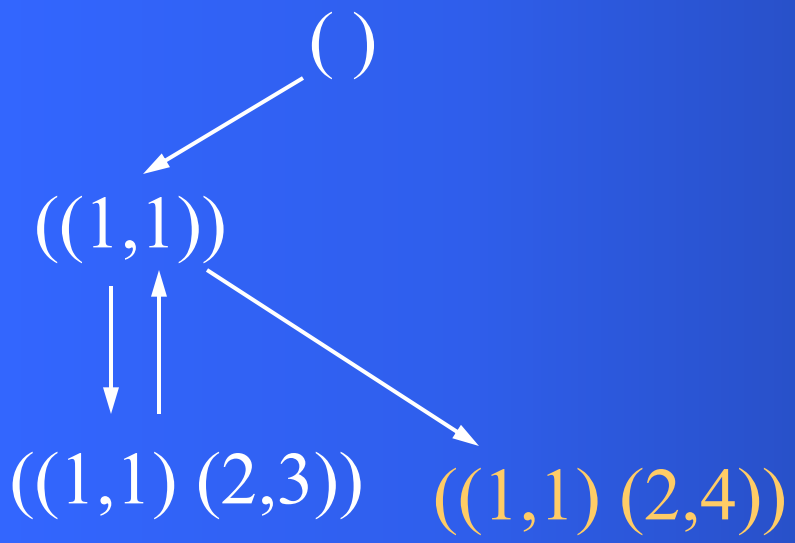


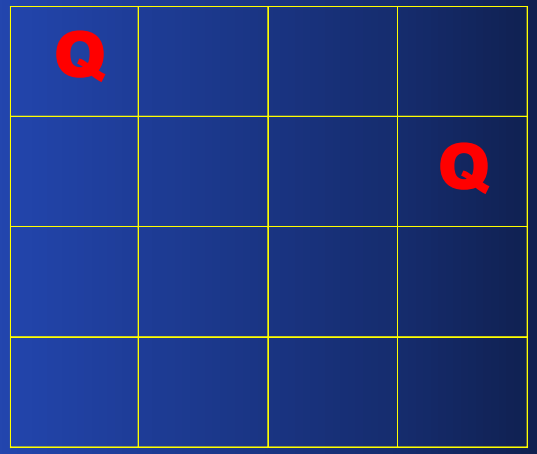
第二层递归:

DATA=(A11)　　Rules=(R24)

R=R24,　　Rules’=NIL

RDATA=(A11　A24)



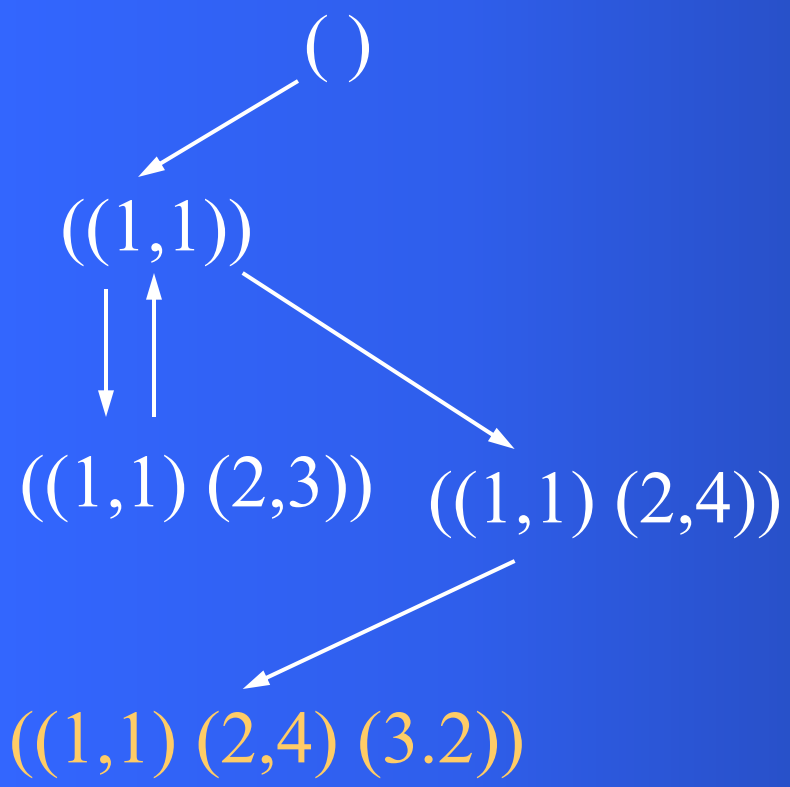


第三层递归:

DATA=(A11, A24)　　Rules=(R32)

R=R32,　　Rules’=NIL

RDATA=(A11 A24 A32)

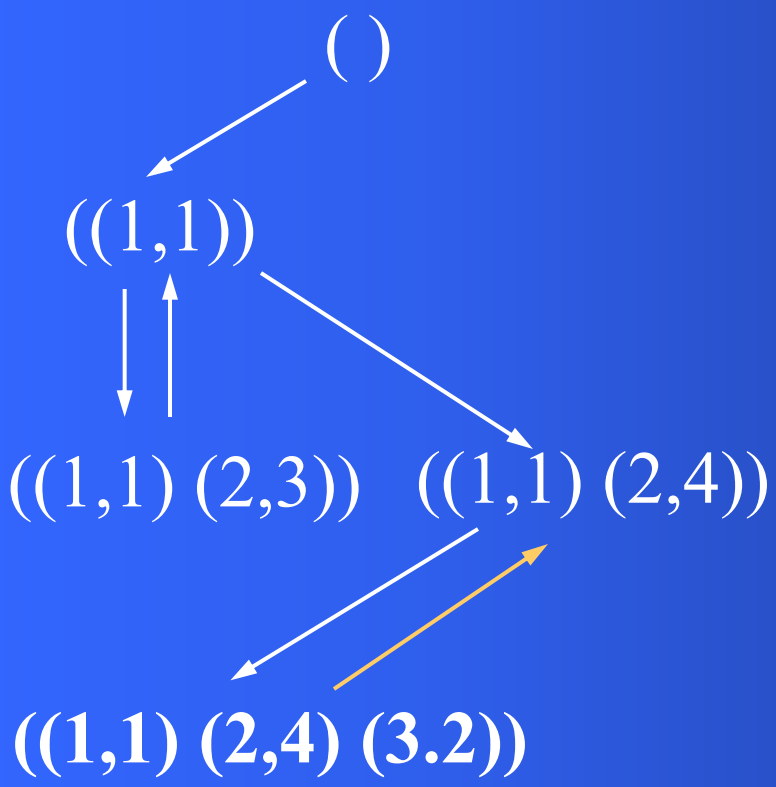


Q			
			Q
	Q		

第四层递归:

DATA=(A11 A24 A32)

Rules=NIL失败,返回第三层递归

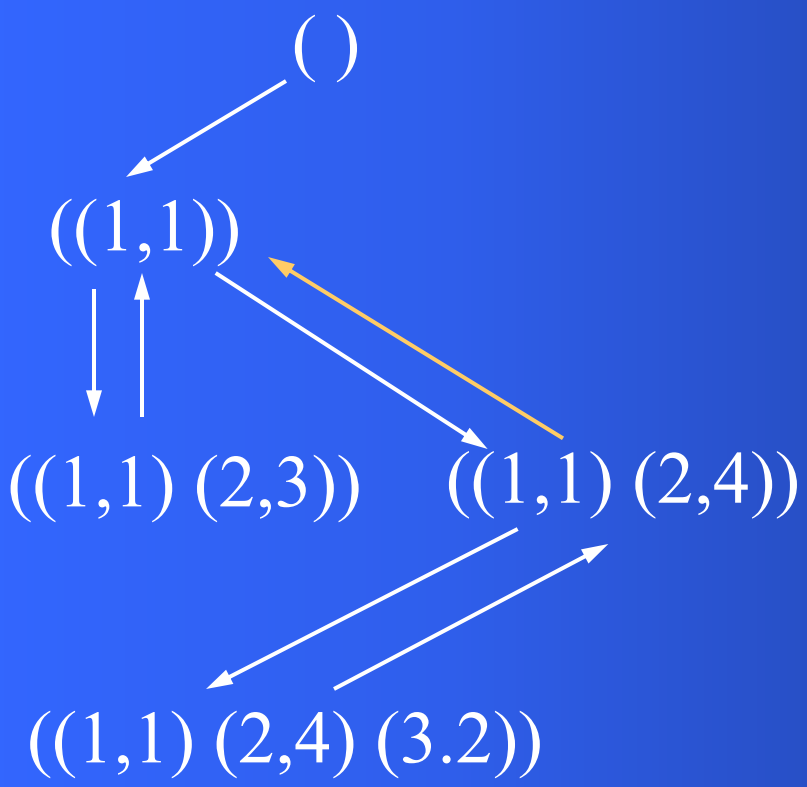


Q			
			Q

第三层递归:

DATA=(A11, A24)

Rules=NIL失败,返回第二层递归

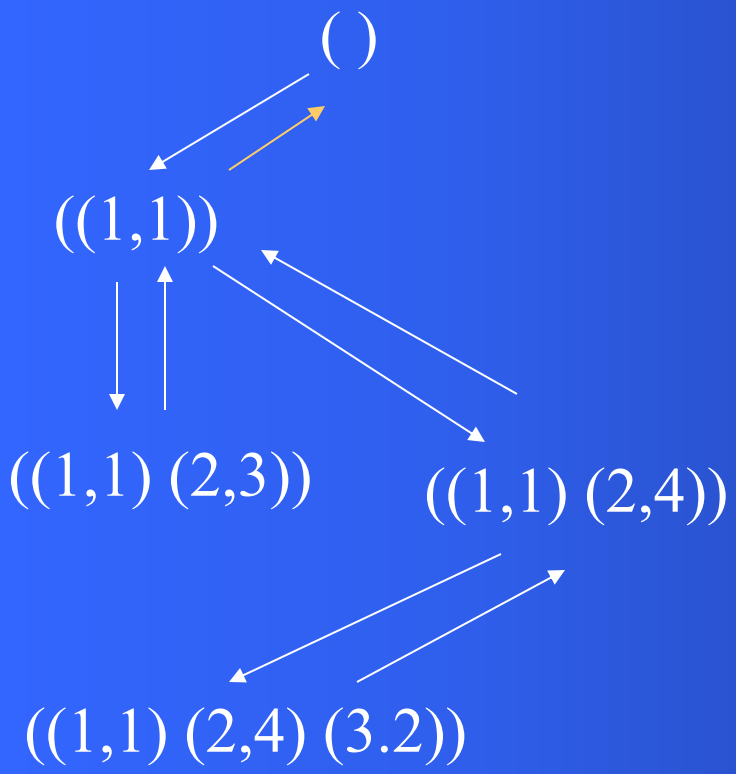


Q			

第二层递归:

DATA=(A11)

Rules=Nil失败,返回第一层递归.



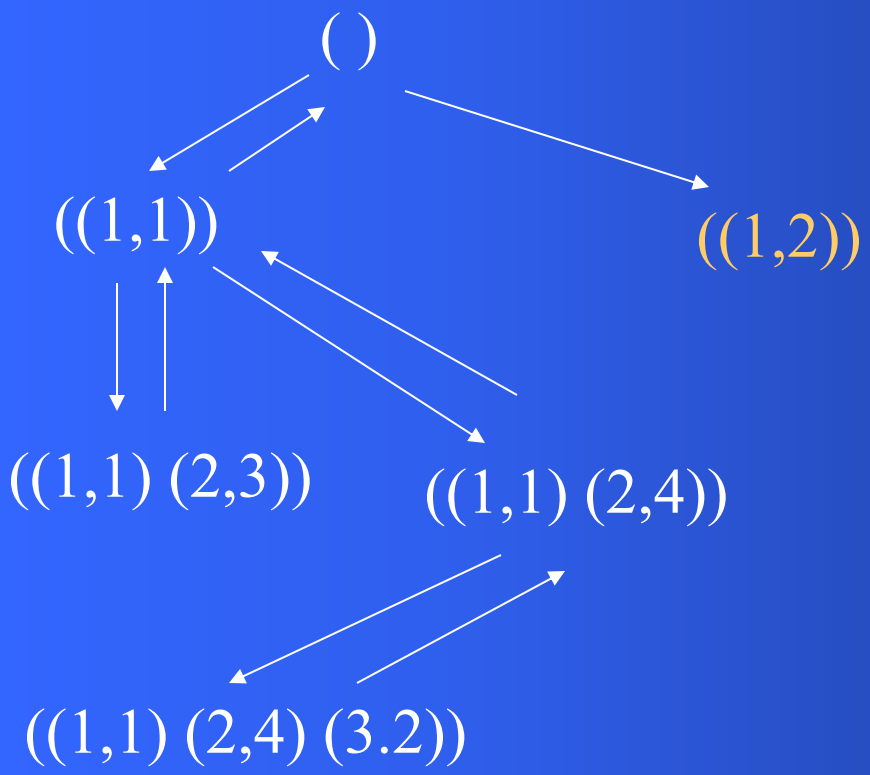


第一层递归:

DATA=( ) Rules=(R12,R13,R14)

R=R12, Rules’=(_R13,R14)

RDATA=(A12)



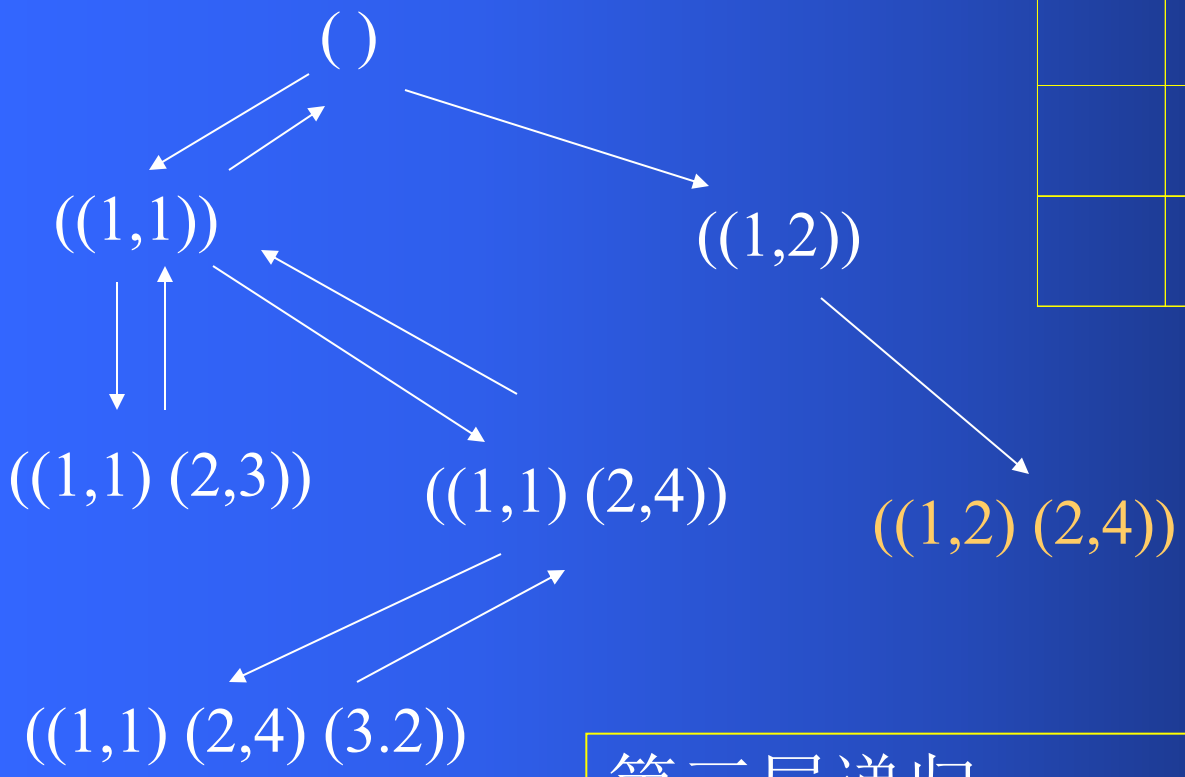
	Q		

第二层递归:

DATA=(A12)　　Rules=(R24)

R=R24　　Rules’=NIL

RDATA=(A12　A24)





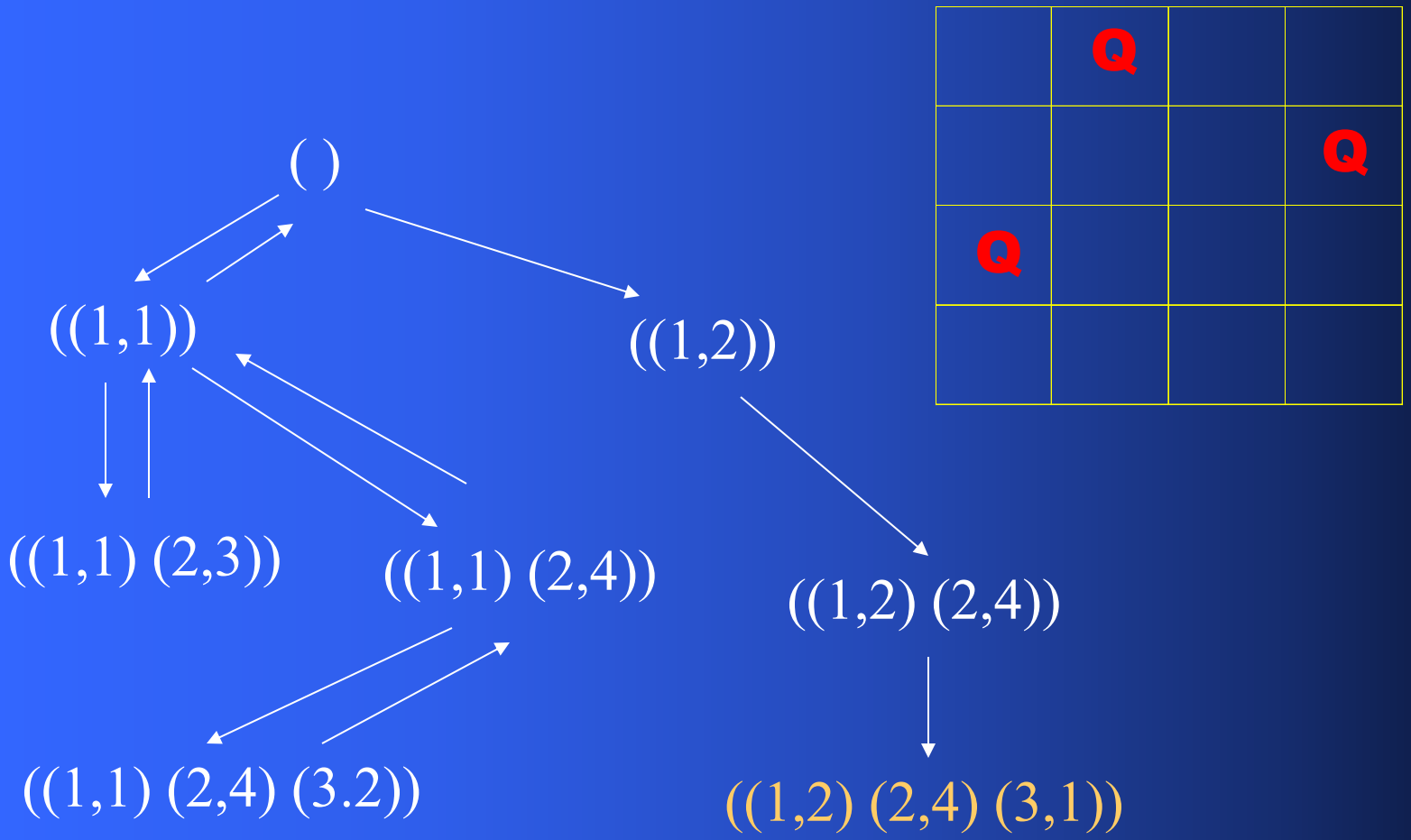
第三层递归:

DATA =(A12　A24)

Rules=(R31)　R=R31

Rules’ =NIL

RDATA=(A12 A24 A31)

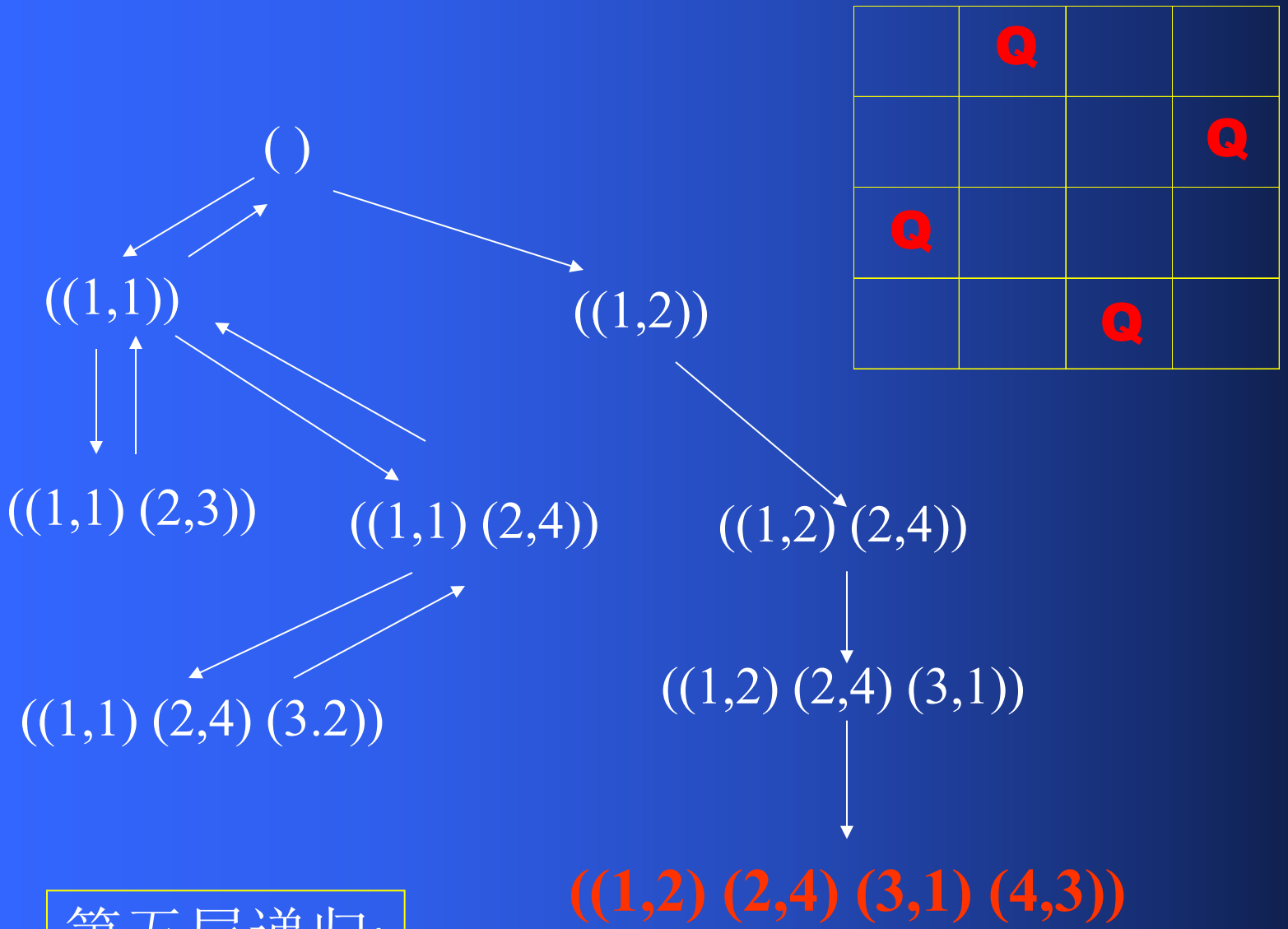


第四层递归:

DATA=(A12 A24 A31) Rules=(R43)

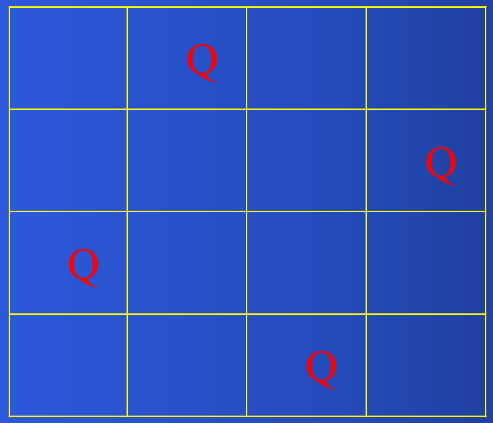
R=R43 Rules’=NIL

RDATA=(A12 A24 A31 A43)



第五层递归:

DATA=(A12 A24 A31 A43) TERM(DATA)=T成功,返回第四层递归 问题的解: PATH=(R12 R24 R31 R43)



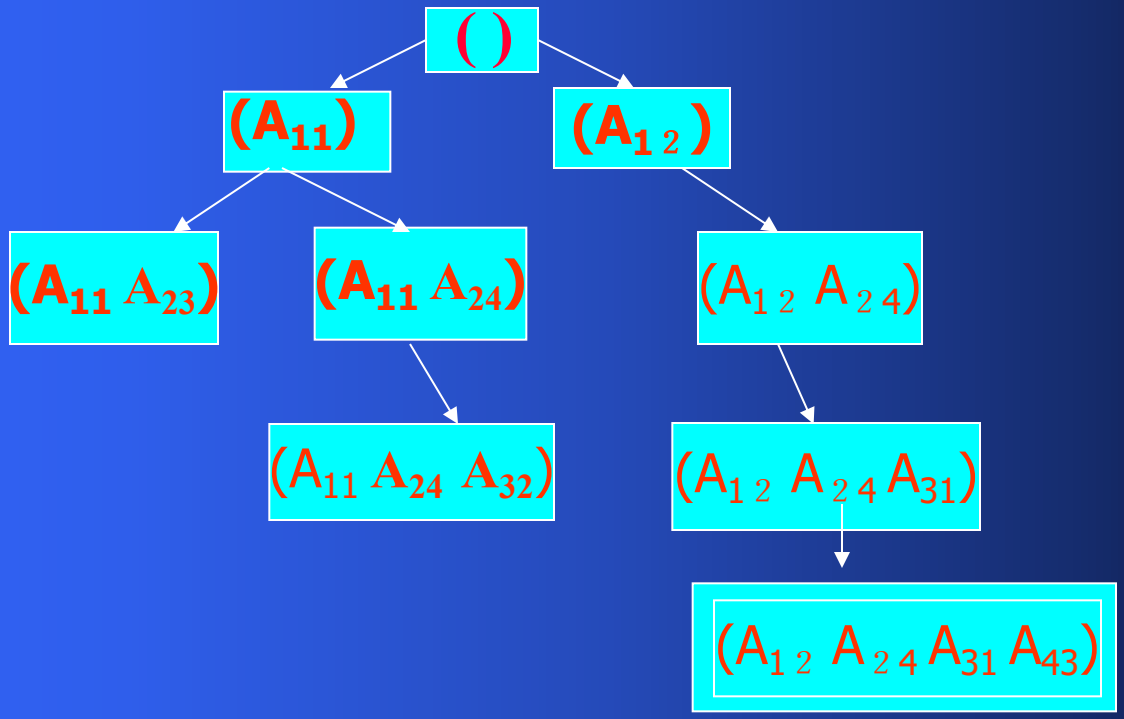


图　解四皇后问题的递归全过程

5.3 产生式系统与图搜索

产生式系统	图搜索
初始事实数据	初始节点
目标条件	目标节点
产生式规则	状态转换规则 问题变换规则
规则库	操作集
动态数据库	节点(状态/问题)
控制策略	搜索策略

表5.1　产生式系统与图搜索对比

可以看出，二者实际是一回事。要说差别的话，图搜索技术描述了问题求解的方法，而产生式系统则给出了实施这种方法的一种系统架构。

这样，问题求解、图搜索和产生式系统三者的关系是：问题求解是目的，图搜索是方法，产生式系统是体系结构形式。

所以，产生式既是一种知识表示的方法，又是一种问题求解的系统架构。

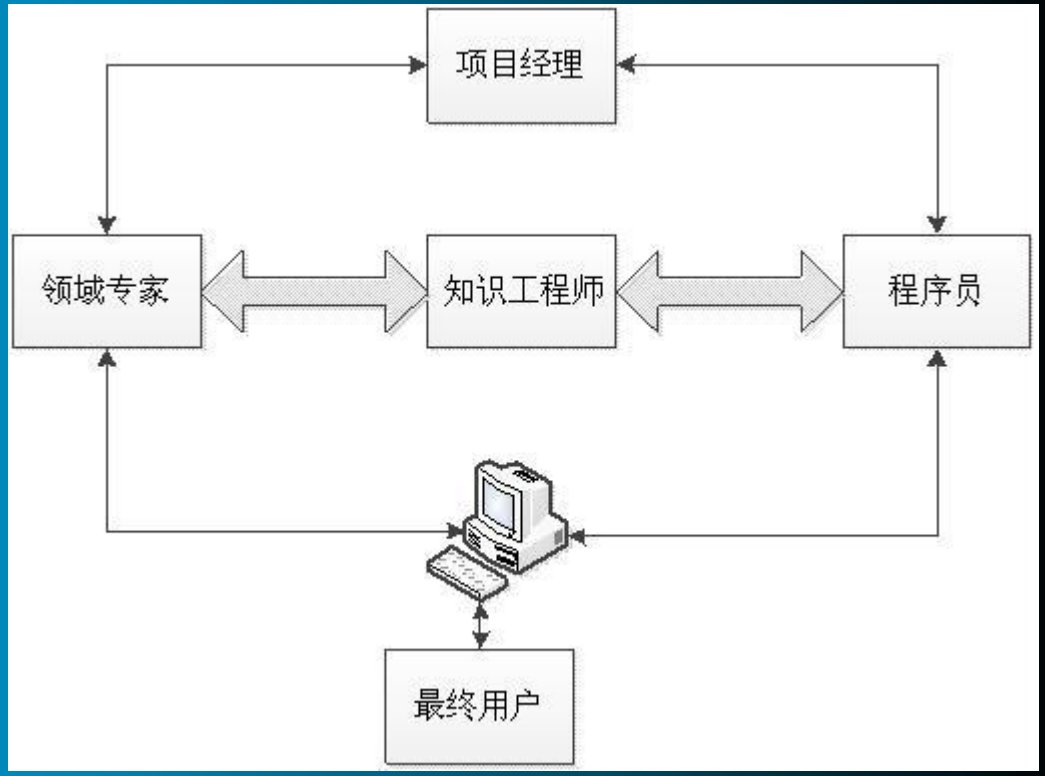
- 产生式系统是一种通用的计算模型，它的真正强大之处是为基于知识的系统提供了一种重要的体系架构。
- 产生式系统的一个有趣的应用是用它对人类认知建模，在20世纪60-70年代，Newell,Simon进行了此项研究，在很大程度上，他们开发的GPS通用问题求解系统奠定了产生式系统在AI中的重要地位。
- 同时，产生式系统所具有的对人类求解问题建模的能力，使他成为设计和建立专家系统的理想工具。在60年代开始成为专家系统最基本的结构。

5.4 基于规则的专家系统

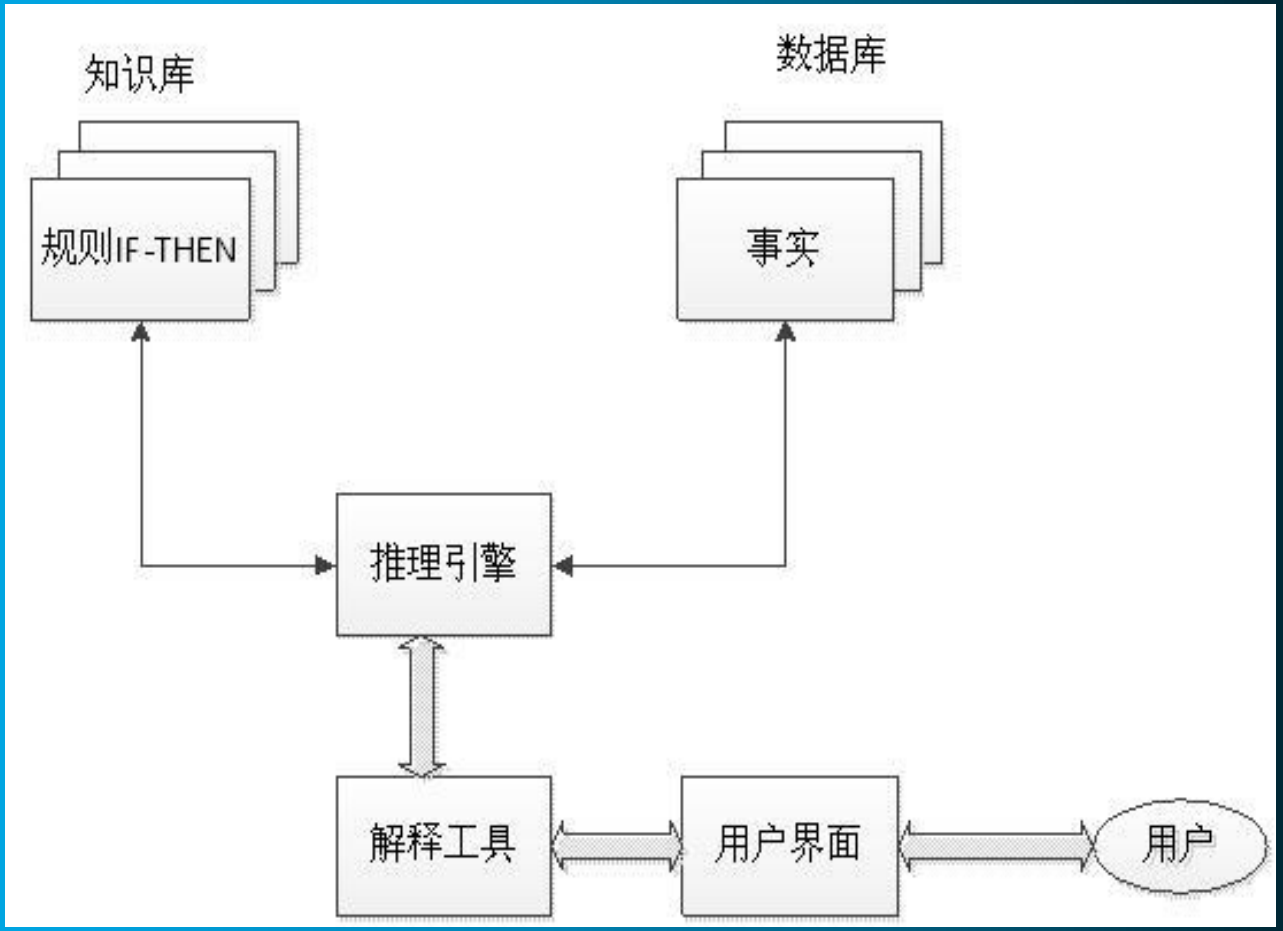
何谓专家系统？

专家系统是一种智能计算机系统，它能使用某一问题领域的知识为为该领域提供专家级的服务，能部分或全部地代替领域专家解决本领域的高难度问题。

专家系统：根据领域的系统知识建立起来的推理系统。它包含五种人：领域专家，知识工程师，程序员，项目经理，最终用户，团队关系图如右图



- 基于规则的专家系统包含五部分：知识库，数据库，推理引擎，解释工具和用户界面。



专家系统结构

